

TD 3 : Apprentissage supervisé

IUT Lannion - Arthur Hoarau

Septembre 2025

La bibliothèque *numpy* sera utilisée pour représenter les listes ainsi que les jeux de données en *Python*. La package *scikit-learn* sera également utilisé pour l'apprentissage automatique. Les exercices sont à effectuer les un après les autres, ne passez à l'exercice suivant que si vous avez terminé le précédent. Pensez à utiliser la *feuille de triche* fournie en annexe qui vous aidera.

1 Les K plus proches voisins (*KNN*)

Exercice 1

Programmez une méthode *distance(x1,x2)* qui calcule et retourne la distance euclidienne entre deux vecteurs *x1* et *x2* de taille *n*. La formule est la suivante :

$$\sqrt{\sum_{i=1}^n (x1_i - x2_i)^2}$$

Exercice 2

Programmez une méthode *voisins(x,dataset)* qui calcul la distance entre *x* et les vecteurs de *dataset* et qui retourne la liste triée des indices de *dataset*, du plus proche au plus éloigné de *x*. Vous pouvez tester votre méthode : *voisin(np.array([1, 2]), np.array([[1, 1], [3, 7], [2, 2]]))* doit retourner : *[0, 2, 1]*.

Exercice 3

Chargez le jeu de données IRIS dans une seule variable *dataset*. Décrivez chacune des variables, pour ce faire indiquez son nom, son type, s'il s'agit d'une variable quantitative indiquez sa moyenne et ses valeurs min et max, s'il s'agit d'une variable qualitative indiquez toutes les valeurs possible ou une description si le nombre de valeurs est trop grand. Chargez cette fois le jeu de données dans deux *numpy.array*, *X* et *y*.

Vérifiez les distributions de votre jeu de données, la valeur de retour pour *X* doit être (150, 4), soit 150 observations sur 4 variables, et celle de *y* doit être

(150), soit 150 observation pour une seule variable.

Exercice 4

Découpez votre jeu de données en un jeu d'entraînement et un jeu de test. Programmez une méthode *vote*(*K*, *voisins*, *y*) qui prend en paramètre un nombre de voisins de *K*, une liste d'indices (la sortie de la méthode *voisins*) et les vraies classes des observations *y*. La méthode *vote* doit retourner la classe retenue après vote majoritaire des *K* plus proches voisins.

Exercice 5

Appliquez la méthode des *K* plus proches voisins sur le jeu de données IRIS. Pour ce faire réalisez la prédiction de votre modèle avec *K* = 3 voisins pour la première observation du jeu de test: *X_{test}*[0]. Pour ce faire utilisez :

```
voisins(X_test[0], X_train)
vote(3, voisins, y_train)
```

Vérifiez si votre prédiction correspond à la classe de la première observation : *y_{test}*[0]. Réalisez maintenant la prédiction de toutes les observations du jeu de test. Quelle est l'exactitude (*accuracy*) du modèle ?

$$Exactitude = \frac{Nombre_prediction_correctes}{Nombre_total_de_prediction}$$

Exercice 6

Utilisez maintenant la méthode des *k* plus proches voisins de la librairie *skit-learn*. Chargez un *KNeighborsClassifier* avec *K* = 3 voisins et affichez son score d'exactitude sur le jeu de test.

Les données doivent être standardisées (centrées et réduites) avant l'appel des *K* plus proches voisins. Standardisez vos données et relancez l'expérience. Obtenez-vous de meilleur performances ? Pourquoi ?

Essayez 5 valeurs de *K* différentes pour le nombre de voisins, quelle semble être la meilleure valeur ?

2 Arbre de décision

Exercice 1

Chargez le jeu de données IRIS et initialisez un *DecisionTreeClassifier*(*criterion* = "entropy") pour choisir l'entropie comme critère de séparation. Comparez les arbres de décision avec les deux critères, que remarquez-vous ?

Pour un même critère l'arbre généré est-il toujours le même ? Pourquoi ?

Exercice 2

Évaluez les performances de votre modèle sur le jeu de test des données IRIS.

Exercice 3

Chargez maintenant le jeu de données *BreastCancer*. décrivez ce jeu de données.

Répétez les exercices 1 et 2 sur ce jeu de données.

3 Comparaison

Pour les deux jeux de données *IRIS* et *BreastCancer* programmez 10 tirages de séparation en jeu de test et jeu d'entraînement. Faites une moyenne des performances des K plus proches voisins et d'un arbre de décision pour chaque jeux de données. Quel est le modèle qui a les meilleurs performances ? (Précisez K et le critère de séparation de l'arbre).